

木幅学習システムの構築

高効率なグラフ解析に向けた木幅概念の学習システム構築

明治大学理工学部情報科学科
小林紹子 玉木久夫 井口幸洋

March 16, 2026



1. はじめに
2. 木と閉路
3. 木分解 (tree decomposition)
4. 木幅 (treewidth)
5. 木幅学習システムの開発
6. 今後の課題
7. まとめ
8. 参考文献

はじめに

はじめに

- グラフ理論 [10] は, ディペンダブルコンピューティング分野における信頼性解析など, 多くの分野で基礎理論として用いられる.
- 一般に, 解くことが困難な問題であっても, 対象となるグラフが木構造であれば, 容易に解けることが知られている.
- 木分解, 木幅はグラフ理論やアルゴリズム設計で重要な概念である.
- 木分解・木幅の認知度が低い原因
 - 理解が難しい (定義が具体的にイメージしにくい, 木分解の構築の難しさ) .
 - 日本語で学べる教材や可視化・インタラクティブな教材が少ない.
- ⇒ 木幅や木分解の定義理解を促進する学習システムを開発

木と閉路

木と閉路

- 閉路：
始点と終点一致し、それ以外の頂点を重複せずに通る道。
- 木：
閉路を含まず、連結である無向グラフ [10].
- 木の特徴
 - 任意の2頂点間にただ一つの単純路が存在 [1].
 - 任意の辺を一つ削除すると、木は複数の連結成分に分裂.

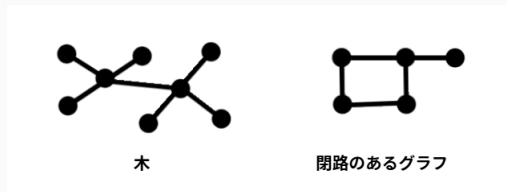


Figure 1: 木と閉路

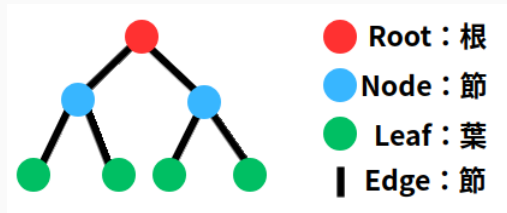


Figure 2: 木構造の用語

木構造はなぜ扱いやすいか

- 木は閉路を持たず、構造が単純.
- 頂点や辺を削除すると、自然にグラフが分割される.
- $\Rightarrow$ 再帰構造を持ち、動的計画法が適用しやすい.
- 一方、一般のグラフは閉路を含み、木と同様のアルゴリズムの適用は困難.

一般グラフを「木に近い構造」として扱う枠組み $\Rightarrow$ 木分解 [7]

- **起源:** 1976 年に Halin が S-functions を定義 [2]. 無限グラフの端を分離する構造として、現在の木分解と実質的に同等の概念を初めて提唱.
- **確立:** 1986 年に Robertson & Seymour が 木分解 として再定義. グラフ・マイナー定理の証明に導入し、同時に NP 困難な問題へのアルゴリズム的有用性を確立.

[7] N. Robertson and P. D. Seymour, "Graph minors. II. Algorithmic aspects of tree-width," *Journal of Algorithms*, vol. 7, no. 3, pp. 309–322, 1986.

木分解 (tree decomposition)

木分解 (tree decomposition) とは

木分解：

一般のグラフの頂点集合を部分集合の集まりとして分割し, 木構造で表現する.

木分解の定義

グラフ $G = (V, E)$ に対し, 木 $T = (I, F)$ とバッグと呼ばれる部分集合 $\{X_i \mid i \in I\}$ が次を満たすとき, これを**木分解**という [7].

(TD1) すべての頂点はどこかのバッグに含まれる.

(TD2) すべての辺の両端は同じバッグに含まれる.

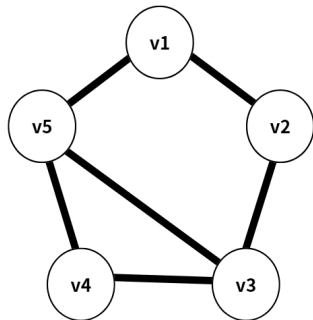
(TD3) 同じ頂点を含むバッグは木 T 上で連結である.

木分解の例

(TD1) すべての頂点はどこかのバッグに含まれる.

(TD2) すべての辺の両端は同じバッグに含まれる.

(TD3) 同じ頂点を含むバッグは木 T 上で連結である. (点線)



5頂点のグラフ

Figure 3: 例

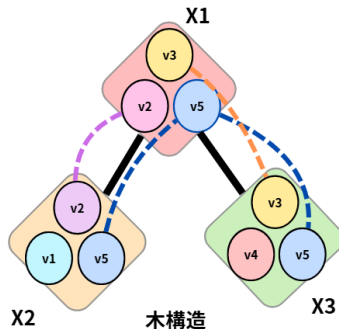


Figure 4: 左図のグラフの木分解の例

木幅 (treewidth)

木幅 (treewidth) とは

- 木分解において重要なのは「部分集合の大きさ」＝「バッグに入れた頂点の個数」.
- この大きさを「幅 (width)」という.
- バッグが小さいほど、木に近い構造を持つと考えられる.

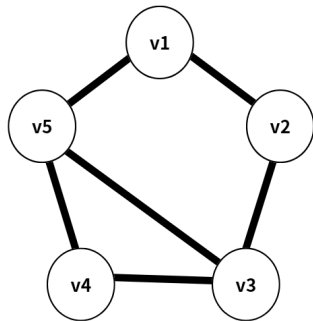
木分解の幅の定義

$$\max_{i \in I} (|X_i| - 1)$$

木幅の定義

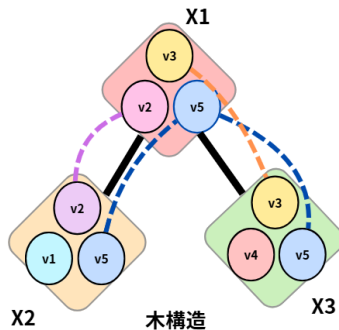
すべての場合における木分解の幅の最小値. (※木の木幅は 1)

木分解と木幅の例



5頂点のグラフ

例（再掲）



木構造

左図のグラフの木分解の例（再掲）

各バッグの大きさ：3

幅： $3 - 1 = 2$

⇒ 他の木分解も考慮した結果, このグラフの木幅は2となる.

木幅学習システムの開発

設計方針

- 木分解の定義や性質を「問題形式」で確認.
- 理解度を即時に確認できる構成.
- 問題の難易度を段階的に上げ, 定義理解から応用へ.
- グラフや木構造をインタラクティブに選択可能.

特徴

- 木分解の構築を直接求めず, 定義理解に必要な前提知識を段階的に獲得.
- 木構造の頂点・辺情報を保持するため, グラフを画像でなくツールで生成.

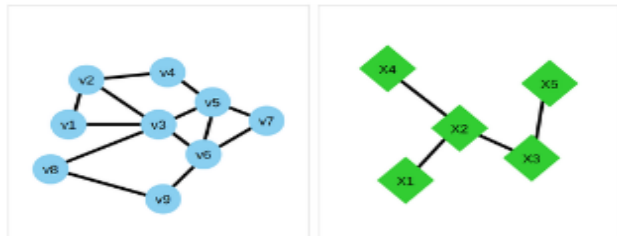
使用技術

- **Node.js**：サーバ側で JavaScript を実行 [5].
- **TypeScript**：型付けによる安全性向上 [4].
- **Next.js / React**：動的で管理しやすい UI 構築 [9, 12, 3, 13].
- **SQLite + Prisma**：軽量 DB と ORM によるデータ管理 [8, 6].
- **ReactFlow**：グラフを直感的に描画・操作可能 [11].

実装済み機能

- 問題提示 (○ × ・ 入力 ・ 選択)
- 解答 ・ 正誤判定
- 解説表示
- グラフのインタラクティブ操作

学習システム一例（入力問題）



以下のバッグ割り当ては木分解の条件2を満たしていない。いくつかの辺が足りていないか。半角数字で答えよ。

- $X1 = \{v1, v2, v3\}$
- $X2 = \{v2, v3, v4, v5, v6\}$
- $X3 = \{v3, v4, v6\}$
- $X4 = \{v5, v6, v7\}$
- $X5 = \{v3, v8, v9\}$

※条件2：グラフGの辺を構成する頂点は必ず同じバッグに含まれる。

半角数字で入力

送信

Figure 5: 入力問題の例

学習システム一例（問題追加（グラフ描画））

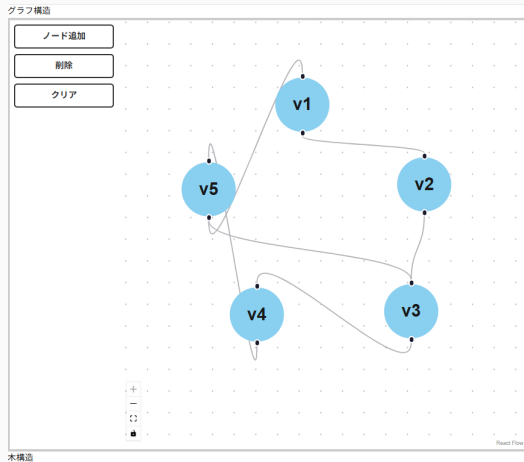


Figure 6: グラフ描画

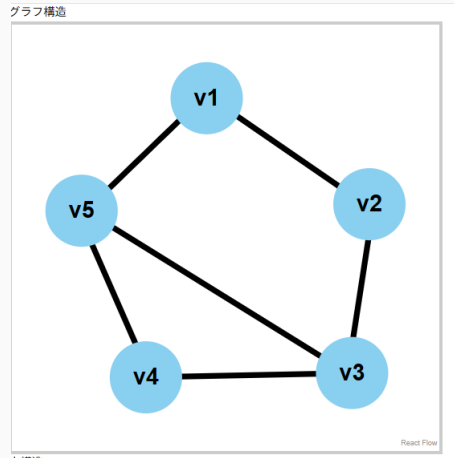


Figure 7: 確認画面

今後の課題

今後の課題

問題・グラフの自動生成

問題内容やグラフ構造を自動的に生成し、学習の多様性を向上



理解度分析

学習履歴を用いて、理解度やつまずきを分析



動機付けの強化

ゲーム要素を導入し、学習意欲の向上を図る



まとめ

- 木幅・木分解は、一般グラフを「木に近い構造」として捉えるための重要な概念である.
- 定義理解の難しさに着目し、問題演習とインタラクティブ操作を通じた学習支援システムを構築した.
- 視覚的・操作的な学習により、木幅・木分解の直感的理解を支援できると考えられる.

参考文献

- [1] John Adrian Bondy, Uppaluri Siva Ramachandra Murty, et al. *Graph theory with applications*, volume 290. Macmillan London, 1976.
- [2] Rudolf Halin. S-functions for graphs. *Journal of geometry*, 8(1-2):171–186, 1976.
- [3] Inc Meta Platforms. React reference overview. <https://ja.react.dev/reference/react>, 2026. Accessed: 2026-2-11.
- [4] Microsoft. Typescript for the new programmer. <https://www.typescriptlang.org/docs/handbook/typescript-from-scratch.html>, 2026. Accessed: 2026-2-11.
- [5] OpenJS Foundation and Node.js contributors. Node.js® とは. <https://nodejs.org/ja/about>, 2026. Accessed: 2026-2-11.
- [6] Inc. Prisma Data. What is prisma orm? <https://www.prisma.io/docs/orm/overview/introduction/what-is-prisma>, 2025. Accessed: 2026-2-11.

- [7] Neil Robertson and Paul D. Seymour. Graph minors. ii. algorithmic aspects of tree-width. *Journal of algorithms*, 7(3):309–322, 1986.
- [8] SQLite Consortium. About sqlite. <https://www.sqlite.org/about.html>, 2025. Accessed: 2026-2-11.
- [9] Inc. Vercel. Next.js docs. <https://nextjs.org/docs>, 2025. Accessed: 2026-2-11.
- [10] Robin J Wilson. *Introduction to graph theory*. Pearson Education India, 1979.
- [11] xyflow team. Overview: Terms and definitions. <https://reactflow.dev/learn/concepts/terms-and-definitions>, 2026. Accessed: 2026-2-11.
- [12] 手島 拓也 他. *TypeScript と React/Next.js でつくる実践 Web アプリケーション開発*. 技術評論社, 2022.
- [13] 山田 祥寛. 改訂新版 これからはじめる *React* 実践入門コンポーネントの基本から *Next.js* によるアプリ開発まで. SB クリエイティブ, 2025.